M1.1: eBPF Kernel Program Types & Loading Pipeline

Theory: Programmable dynamic attachments hooking across Linux kernel network layers. eBPF hooks allow safe bytecode injections at runtime.

Details: Cilium deploys ‘XDP‘ at driver levels for fast drops, ‘tc-bpf‘ at traffic control layers (ingress/egress) for pod routing, and ‘sockops‘ at TCP socket stacks for connection shortcuts. The Cilium Agent loads bytecodes via ‘bpf()‘ system calls and pins them to ‘sys/fs/bpf‘.

Trade-off: Dynamic loading blocks execution and creates host CPU spikes. Restrict compile loads to network topology changes and rely on runtime BPF Maps for routing states.

M1.2: LLVM/Clang eBPF Machine Code Generation

Theory: Compilation pipeline translating C subsets to constrained eBPF ISA virtual instruction blocks.

Details: Clang translates restricted C files into 64-bit eBPF bytecode. Under kernel constraints, loops must be unrolled, stack variables must not escape, and compilation sizing is checked prior to load.

Trade-off: Standard libraries are not supported. Developers must rely on ‘bpf_helper‘ utilities for kernel calls. Complex computations are handled by the Go agent.

M1.3: libbpf & bpftool Kernel Verification

Theory: The kernel safety verifier analyzing control flows to prevent buggy code from freezing or crashing Linux hosts.

Details: When bytecodes are loaded, the kernel ‘Verifier‘ walks all assembly pathways, checking for infinite loops, out-of-bounds register reads, and null pointer dereferences. ‘libbpf‘ handles relocations and ‘bpftool‘ audits runtime maps.

Trade-off: Strict validation rules block complex logic. Developers must insert explicit boundary checks (e.g. ‘if (ptr + 1 > data_end) return tc_drop;‘) to prove code safety.

M1.4: CO-RE & BTF Relocation Mechanism

Theory: Relocation system allowing eBPF binaries to run across different Linux kernel versions.

Details: eBPF structures depend on kernel member offsets. CO-RE compares compiled BTF (BPF Type Format) definitions with host kernel BTF formats at loading, dynamically patch-adjusting instruction offsets.

Trade-off: Requires host kernels compile with ‘CONFIG_DEBUG_INFO_BTF‘. Legacy machines lack BTF support, forcing local compiler setups for on-the-fly compilation.

M1.5: BPF Map Types & Kernel Hash Tables

Theory: Thread-safe shared memory maps enabling state sharing between eBPF programs and user-space agents.

Details: BPF Maps hold data states. Cilium utilizes ‘BPF_MAP_TYPE_HASH‘ for conntrack tables, ‘BPF_MAP_TYPE_LRU_HASH‘ for session mappings, and ‘BPF_MAP_TYPE_LPM_TRIE‘ for IP prefix matches. Read/writes are handled via lock-free helpers.

Trade-off: Map sizing is allocated at compilation. Burst traffic exceeding conntrack maps leads to drops. Cilium pre-allocates large map slots and runs background cleanup sweeps.

M1.6: Cilium Agent Dynamic Compilation Flow

Theory: Dynamic compilation engine generating C headers and binaries matching real-time Kubernetes states.

Details: The Cilium Agent daemon tracks K8s API events. When network changes occur, the Agent updates constants (e.g. local IP ranges) in dynamic header files and compiles targeted ELF outputs via ‘clang‘.

Trade-off: Compiling Clang repeatedly under cluster scaling events hogs host CPU cores. Cilium utilizes static caching and compilation debouncing to optimize load times.

M2.1: XDP Driver-Level Fast Packet Interception

Theory: Ingests and processes packets before memory allocations occur to bypass OS networking pipelines.

Details: XDP runs inside network drivers at the DMA buffer stage. Before allocating ‘sk_buff‘ structures, eBPF code drops spam packets (‘XDP_DROP‘) or forwards packets to other adapters (‘XDP_REDIRECT‘).

Trade-off: Missing packet skb references prevents using advanced kernel network helpers. Cilium uses XDP for DDoS protection and routes core payloads to tc-bpf.

M2.2: tc (Traffic Control) Hook Infrastructure

Theory: Network protocol hook executing packet logic inside Linux kernel sk_buff structures.

Details: tc-bpf hooks run inside ingress and egress scheduler queues. They analyze the complete ‘sk_buff‘ reference, using routing table index modifications and packet encapsulation helpers.

Trade-off: Runs slightly later than XDP, consuming skb memory allocation overhead. Cilium uses tc as its main routing datapath for Kubernetes pods.

M2.3: eBPF Lock-Free Packet Parsing

Theory: Fast packet parser computing protocol offsets via lock-free pointer algebra.

Details: eBPF routines receive start (‘data‘) and end (‘data_end‘) packet memory pointers. Code parses headers by cast-offsetting address locations: ‘struct iphdr *ip = (data + eth_len)‘. The verifier forces validation checks before read access.

Trade-off: Strict pointer boundary requirements require verbose bounds-checking assertions throughout parser codes.

M2.4: Conntrack Connection Tracking BPF Maps

Theory: Kernel-level connection tracking engine mapping concurrent connection states.

Details: Cilium stores connection mappings using five-tuple keys ‘(SrcIP, SrcPort, DstIP, DstPort, Proto)‘ in hash maps. Ingress paths validate SYN states, creating tracking entries to speed up return packets.

Trade-off: Requires custom timeout cleanup rules. The Cilium Agent runs background garbage collection sweeps to purge stale keys and avoid map overflows.

M2.5: eBPF Kernel NAT & L3 Routing Rewriting

Theory: Fast header rewriting and direct network card redirects bypassing standard OS routing tables.

Details: Under NAT redirection, eBPF code directly updates IP/Port coordinates inside ‘sk_buff‘, recalculating L3/L4 checksums. The packet is then routed directly via ‘bpf_redirect‘ to the target interface.

Trade-off: Hardware checksum offloading failures cause drops. Cilium checks adapter features and falls back to CPU-based checksum recalculation when needed.

M2.6: VxLAN/Geneve Tunnel Encapsulation & Decapsulation

Theory: Overlay networking encapsulate and decapsulate payloads to bridge communication across hosts.

Details: For cross-host routes, tc-bpf appends outer host IPs and VxLAN/Geneve headers to payloads. Destination hosts capture outer packets, strip overlay headers, and route original packets to target pods.

Trade-off: Tunnel packaging adds 50-byte headers. Exceeding physical MTU limits prompts packet fragmentation. Set pod network interfaces MTU sizes to 1450.

M3.1: sockops Socket-Layer Event Interception

Theory: Intercepts TCP handshake events to manage sockets at the connection phase.

Details: Deployed at ‘BPF_PROG_TYPE_SOCK_OPS‘. As sockets complete handshakes, BPF code extracts IP/Port coordinates and gathers active socket context pointers.

Trade-off: Sockops tracking primarily optimizes local loopback traffic. Adding cross-host connections consumes map memories with no execution performance returns.

M3.2: SOCKMAP Descriptor Binding Hash Maps

Theory: Specialized BPF Map linking socket file descriptors to high-performance redirection indexes.

Details: Wasmtime registers active local TCP sockets to ‘BPF_MAP_TYPE_SOCKMAP‘ under ‘(IP, Port, DstIP, DstPort)‘ keys, building direct connection references.

Trade-off: Connection terminations (FIN/RST) must clear registers to prevent dangling pointers. Cilium hooks socket state changes to drop mappings dynamically.

M3.3: bpf_msg_redirect_hash Send/Receive Buffer Bypass

Theory: Direct socket-to-socket memory shortcutting bypassing the entire Linux network stack.

Details: Hooked at ‘BPF_PROG_TYPE_SK_MSG‘. As Pod A calls ‘sendmsg‘, eBPF intercepts payloads in the send buffer, queries the SOCKMAP for Pod B’s socket, and writes data directly to Pod B’s receive queue.

Trade-off: Bypassing TCP/IP stacks hides payloads from standard ‘tcpdump‘ diagnostics. Hubble telemetry is required to capture and trace local traffic.

M3.4: Local Pod-to-Pod Communication Speedups & Trade-offs

Theory: Trade-off modeling evaluating local loopback performance boosts against observability gaps.

Details: Skipping system loops, software interrupts, and kernel routing layers maximizes network throughput and drops latency.

Trade-off: Bypasses classic host firewall rule engines. Security auditing must migrate to eBPF identity label tables.

M3.5: Socket Lock Contention & Syscall Elimination

Theory: Optimizes multi-core network operations by reducing CPU lock contention and context switches.

Details: eBPF redirection reduces TCP stack processing, avoiding locks on network cards and eliminating soft interrupt threads.

Trade-off: Shortcut mechanisms primarily support TCP traffic; UDP acceleration yields lower performance returns.

M3.6: Unix Domain Socket Performance Comparisons

Theory: Compares code refactoring costs to eBPF-accelerated TCP network loops.

Details: Standard Unix Domain Sockets outpace normal TCP loops. Cilium’s TCP shortcut achieves UDS-level performance without code changes, keeping client URLs (‘http://ip:port‘) intact.

Trade-off: For small payloads, UDS is faster due to lack of handshake routines. Under high-throughput workloads, the performance gap is negligible.

M4.1: Maglev Lookup Table Construction & Hash Allocation

Theory: Consistent hashing algorithm mapping requests to backend endpoints with minimal connection drops under scaling.

Details: Maglev computes permutation sequences for backend pods to occupy slots in a large lookup table (e.g. size 65537). The resulting routing lookup map is stored in a BPF Map.

Trade-off: Recomputing lookup tables requires CPU cycles. Cilium Agent performs allocations in user-space and updates the BPF Map.

M4.2: Load Balancing Virtual IP Map Routing

Theory: Rewrites Kubernetes Service VIP targets to target Pod IP addresses inside kernel ingress paths.

Details: As VIP packets enter, eBPF load-balancing logic selects backends using the Maglev lookup map, rewriting the destination IP to the real Pod IP.

Trade-off: Returns must route back through the gateway to reverse NAT the source IP to the VIP, otherwise clients reject mismatched addresses.

M4.3: BPF LRU Map Session Affinity Tracking

Theory: Track and direct client requests to sticky backend pods using high-speed LRU tables.

Details: Session connections are logged in ‘BPF_MAP_TYPE_LRU_HASH‘ under ‘(ClientIP, ServiceVIP) -> PodIP‘ mappings. The kernel LRU algorithm purges cold entries dynamically to prevent OOM.

Trade-off: LRU eviction operates probabilistically. Under high collision rates, session affinity can fail, requiring application-level retries.

M4.4: Direct Server Return Routing Mode

Theory: Accelerates load balancers by allowing backend pods to respond directly to clients, bypassing gateway nodes.

Details: The load balancer encodes the client VIP inside VxLAN/IPv4 option headers without updating destination IPs. Backend pods unpack headers and write responses using the VIP as the source address.

Trade-off: Path routers must support asymmetric paths, and routers must not drop source-mismatched MAC addresses.

M5.1: Identity-Based Security Label Model

Theory: Decouples network firewall rules from dynamic pod IP addresses by assigning static security labels.

Details: Cilium groups pods by labels, assigning them a 32-bit ‘Identity‘ integer. Overlay packet headers embed the sender’s identity. Receivers validate identity permissions against policy maps.

Trade-off: Identity allocations depend on centralized controller consensus. Cilium reserves fallback tags to keep systems secure during API splits.

M5.2: IP-to-Identity Address Hash Map Resolution

Theory: High-speed lookup structures linking IP prefixes to security identity numbers.

Details: Under direct routing setups, Cilium manages IP-to-Identity mappings using ‘BPF_MAP_TYPE_LPM_TRIE‘ tables to run longest-prefix matches inside kernel ingress passes.

Trade-off: Large clusters create map synchronization costs. Cilium mitigates this by only syncing maps of active local connection paths.

M5.3: L3/L4/L7 Hybrid Security Policy Enforcement

Theory: Layered firewall architecture combining fast kernel packet drops with fine-grained L7 application proxies.

Details: L3/L4 policies are enforced directly inside tc-bpf with minimal cost. L7 rules (e.g. HTTP PATH) redirect TCP streams to a local Envoy instance for deep parsing.

Trade-off: L7 audits redirect packets to user-space Envoy, adding context switches. Enforce L3/L4 checks first to drop unwanted traffic.

M5.4: IPsec & WireGuard Kernel-Level Encryption

Theory: Automated encryption pathways protecting pod-to-pod network traffic.

Details: eBPF routes packets and marks them, triggering Linux native IPsec (XFRM) or WireGuard kernel drivers to encrypt outgoing traffic.

Trade-off: Encryption algorithms (e.g. AES-GCM) consume host CPU cycles, decreasing network throughput. Enable hardware accelerated AES-NI instructions to offset costs.

M6.1: Envoy Sidecarless Service Mesh Architecture

Theory: Replaces pod-local sidecars with a single shared Envoy proxy per host node to optimize resources.

Details: Pods route traffic through a shared node-level Envoy proxy. tc-bpf captures port traffic and redirects it to the proxy socket in kernel space.

Trade-off: Reduces host memory overhead. However, host Envoy failures break network traffic for all local pods, requiring highly available monitors.

M6.2: Hubble Traffic Observability & BPF Metadata Tracing

Theory: Cloud-native network observability framework tracking network latency and drops using eBPF data.

Details: Critical tc/XDP pathways write to BPF Ring Buffers on packet drops or policy blocks. The Hubble daemon reads buffers and generates topology views.

Trade-off: Ring Buffer bandwidth is finite. Hubble drops telemetry logs or uses sampling under packet flood conditions to prevent kernel starvation.

Zone T1: eBPF Helper APIs

bpf_map_lookup_elem(), bpf_map_update_elem(), bpf_redirect(), bpf_msg_redirect_hash(), bpf_perf_event_output(), XDP_DROP, XDP_REDIRECT, XDP_PASS

Zone T2: System Network Faults

ebpf_verifier_rejection, conntrack_map_overflow, socket_redirect_dangling_pointer, MTU_exceeded_fragmentation, maglev_hash_collision, identity_label_sync_split